

Linux知识

1.Linux常用命令

1.1 目录处理命令

ls

- -a (all)：显示所有文件及目录
- -l[h] (long)：显示文件各个属性
- -d：显示目录属性
- -i：显示ig点

mkdir

功能描述：创建新目录

-p：递归创建

cd

功能描述：切换目录

- cd .：当前目录
- cd ..：上一级目录

pwd

功能描述：显示当前目录

rmdir

功能描述：删除空目录

cp

功能描述：复制文件或目录

- -r：复制目录
- -p：保留文件属性

mv

功能描述：剪切文件、改名

- **-i**: 若指定目录已有同名文件，则先询问是否覆盖旧文件
- **-f**: 在 **mv** 操作要覆盖某已有的目标文件时不给任何指示;

rm

功能描述：删除文件

- **-r**: 删除目录
- **-f**: 强制执行

1.2 文件处理命令

touch

功能描述：创建空文件

cat

功能描述：显示文件内容

- **-n**: 显示行号

tac

功能描述：显示文件内容（反向列示）

more

功能描述：分页显示文件内容

- （空格）或**f**: 翻页
- （Enter）: 换行
- **q**或**Q**: 退出

less

功能描述：分页显示文件内容（可向上翻页）

- （pageup）: 向上翻页
- （up）: 向上换行
- /（关键词）: 关键词翻白；**-n**: 显示下一个关键词

head

功能描述：显示文件前面几行（默认10行）

- -n: 指定行数

tail

功能描述：显示文件最后几行（默认10行）

- -n: 指定行数
- -f: 动态显示文件末尾内容

1.3 链接命令

ln

功能描述：生成链接文件

- -s: 创建软连接

软链接vs硬链接：

软链接的作用与windows的快捷方式类似

硬链接可以同步更新

1.4 权限管理命令

chmod

功能描述：改变文件或目录权限

chmod [{ugoa}]{+ -=}{rwx}} [文件或目录] [mode=421] []文件或目录

- -R: 递归修改

文件目录权限总结：

代表字符	权限	对文件的含义	对目录的含义
r	读	可以查看文件内容	可以列出目录中的内容
w	写	可以修改文件内容	可以在目录中创建、删除文件
x	执行	可以执行文件	可以进入目录

chown

功能描述：改变文件或目录的所有者

`chown [用户] [文件或目录]`

只有管理员root可以改变

chgrp

功能描述：改变文件或目录的所属组

`chgrp [用户] [文件或目录]`

一个用户可以属于很多组，但是只能有一个缺省组

umask

功能描述：显示、设置文件的缺省权限

- **-S**：以rwx形式显示新建文件缺省权限

1.5 文件搜索命令

find

功能描述：文件搜索

`find [搜索范围] [匹配条件]`

- **-name**：根据文件名搜索，可以用通配符“*”（匹配任意字符），“?”（匹配单个字符）
- **-iname**：不区分大小写根据文件名搜索
- **-size**：根据文件大小搜索，+n 大于，-n 小于，n 等于
- **-user**：根据用户名搜索
- **-a**：两个条件同时满足
- **-o**：两个条件满足任意一个即可
- **-type**：根据文件类型查找。f 文件，d 目录，l 软链接

locate

功能描述：在文件资料库中查找文件，文件资料库会定期更新

- **-i**：不区分大小写搜索

可以使用updatedb可以更新资料库

放在/tmp的临时文件是不会被查找

which

功能描述：搜索命令所在目录及别名信息

whereis

功能描述：搜索命令所在目录及帮助文档路径

grep

`grep -iv [字符串][文件名]`

功能描述：在文件中搜寻字符串匹配的行并输出

- `-i`：不区分大小写
- `-v`：排除指定字符串

1.6 帮助命令

man

功能描述：获得帮助信息

help

功能描述：获得shell内置命令的帮助信息

1.7 用户管理命令

useradd

功能描述：添加新用户

passwd

功能描述：设置用户密码

who

功能描述：查看登录用户信息

`tty`：本地终端

`pts`：远程终端

w

功能描述：查看登录用户详细信息

1.8 压缩解压命令

gzip

功能描述：压缩文件成.gz。压缩比约为5。只能压缩文件，不能压缩目录。不保存源文件。

gunzip

功能描述：解压缩.gz的压缩文件

tar

功能描述：打包目录

- -c: 打包
- -x: 解包
- -v: 显示详细信息
- -f: 指定文件名
- -z: 打包同时压缩

压缩后文件格式：.tar.gz

zip

windows和linux都支持的压缩格式。压缩率为80%

- -r: 压缩目录

upzip

解压.zip的压缩文件

bzip2

压缩比很高

- -k: 产生压缩文件后保留原文件

2.文本编辑器**vim**

2.1 常见操作

命令模式转换成编辑模式

命令	作用
a	在光标所在字符后插入
A	在光标所在行尾插入
i	在光标所在字符前插入
I	在光标所有行行首插入
o	在光标下插入新行
O	在光标下插入新行

定位命令

命令	作用
: set nu	设置行号
: set nonu	取消行号
gg	到第1行
G	到最后1行
nG	到第n行
: n	到第n行
\$	移至行尾
0	移至行首

删除命令

命令	作用
x	删除光标所在字符
nx	删除光标所在行后n个字符
dd	删除光标所在行， ndd 删除n行
dG	删除光标所在行到文件末尾内容
D	删除光标所在处到行尾内容
:n1, n2d	删除指定范围的行

复制和剪切命令

命令	作用
yy	复制当前行
nyy	复制当前行以下n行
dd	剪切当前行
ndd	剪切当前行以下n行
p、P	粘贴在当前光标所在行下或行上

替换和取消命令

命令	作用
----	----

命令	作用
r	取代光标所在处字符
R	从光标所在处开始替换字符，按Esc结束
u	取消上一步操作

搜索和搜索替换命令

命令	作用
/string	搜索指定字符串，搜索时忽略大小写 :set ic
n	搜索指定字符串的下一个出现位置
:%s/old/new/g	全文替换指定字符串
:n1,n2s/old/new/g	在一定范围内替换指定字符串

保存和退出命令

命令	作用
w	保存修改
:w new_filename	另存为指定文件
:wq	保存退出
ZZ	快捷键，保存退出
q!	不保存退出
:wq!	保存退出（文件所有者及root可使用）

2.2 常见技巧

:r 文件名：导入文件内容

:r !命令：导入命令执行结果

:map 快捷键 触发命令：定义快捷键

:n1,n2s/^/#/g：连续行首注释

3.Shell

shell是解释执行的脚本语言，在shell中可以直接调用linux系统命令。

3.1 bash基本功能

3.1.1 历史命令

历史命令默认保存1000条，可以在环境变量配置文件/etc/profile中修改

history [选项] [历史命令保存文件]

- -c：清空历史命令

- `-w`: 把缓存中的历史命令写入历史命令保存文件`~/.bash_history`

`!n`: 重新执行第`n`条命令

`!!`: 重新执行上一条命令

3.1.2 补全

`tab`键补全

3.1.3 别名

`alias 别名='原名'`

3.1.4 快捷键

快捷键	作用
<code>ctrl+c</code>	强制终止当前的命令
<code>ctrl+l</code>	清屏，相当于 <code>clear</code>
<code>ctrl+u</code>	删除或剪切光标之前的命令
<code>ctrl+y</code>	粘贴 <code>ctrl+u</code> 或 <code>ctrl+k</code> 剪切的内容
<code>ctrl+r</code>	在历史命令中搜索
<code>ctrl+d</code>	退出当前终端

3.1.5 输入输出重定向

输出重定向

类型	符号	作用
标准输出重定向	命令 > 文件	以覆盖的方式，把命令的正确输出输出到指定的文件或设备当中
标准输出重定向	命令 >> 文件	以追加的方式，把命令的正确输出输出到指定的文件或设备当中
标准错误输出重定向	错误命令 2> 文件	以覆盖的方式，把命令的错误输出输出到指定的文件或设备当中
标准错误输出重定向	错误命令 2>> 文件	以追加的方式，把命令的错误输出输出到指定的文件或设备当中
正确的输出和错误的输出同时保存	命令 > 文件 2>&1	以覆盖的方式，把命令的正确输出和错误的输出都保存到指定的文件或设备当中
正确的输出和错误的输出同时保存	命令 >> 文件 2>&1	以追加的方式，把命令的正确输出和错误的输出都保存到指定的文件或设备当中
正确的输出和错误的输出同时保存	命令 &> 文件	以覆盖的方式，把命令的正确输出和错误的输出都保存到指定的文件或设备当中
正确的输出和错误的输出同时保存	命令 &>> 文件	以追加的方式，把命令的正确输出和错误的输出都保存到指定的文件或设备当中

类型	符号	作用
正确的输出和错误 的输出同时保存	命令 >> 文件 1 2>> 文件2	把正确的输出追加到文件1中，把错误的输出追加到文件2中

输入重定向

wc [选项] [文件名]

- -c: 统计字节数
- -w: 统计单词数
- -l: 统计行数

命令<文件: 把文件作为命令的输入

3.1.6 多命令顺序执行

多命令执行 符号	作用
;	多个命令顺序执行，命令之间没有任何逻辑关系
&&	逻辑与。当命令1正确执行，则命令2才去执行。当命令1执行不正确，则命令2不会执行
	逻辑或。当命令1执行不正确，则命令2才会执行。当命令2正确执行，则命令2不会执行

管道符

命令1 | 命令2

命令1的正确输出作为命令2的操作对象

例子:

```
ll -a /etc/ | more
```

3.1.7 通配符和其他特殊符号

通配 符	作用
?	匹配一个任意字符
*	匹配0个或任意多个任意字符
[]	匹配中括号中任意一个字符。例如: [abc]代表一定匹配一个字符，或者是a，或者是b，或者是c。
[-]	匹配中括号中任意一个字符，-代表一个范围。例如: [a-z]代表匹配一个小写字母
[^]	逻辑非，表示匹配不是中括号内的一个字符。

其他特殊符号

符号	作用
' '	单引号。在单引号中所有的特殊符号都没有特殊含义
" "	双引号。在双引号中特殊符号都没有特殊含义，但是“\$”、“`”、“\”是例外，拥有“调用变量的值”、“引用命令”和“转义符”的特殊含义
` `	反引号。反引号括起来的内容是系统命令，在bash中会先执行它。和\$()作用一样，不过推荐使用\$()，因为反引号容易看错
\$()	和反引号作用一样，用来引用系统命令。
#	在Shell脚本中，#开头的行代表注释。
\$	用户调用变量的值，如需要调用变量name的值时，需要用\$name的方式得到变量的值。
\	转义符，跟在\之后的特殊符号将失去特殊含义，变为普通字符

3.2 Bash变量

Bash中，变量默认类型为字符串型，如果要进行数值运算，则必须指定变量类型为数值型。

3.2.1 用户自定义变量

变量定义

```
name="shen chao"
```

变量叠加

```
aa=123
```

```
aa="$aa"456
```

```
aa=${aa}789
```

变量调用

```
echo $name
```

变量查看

```
set
```

变量删除

```
unset name
```

3.2.2 环境变量

用户自定义变量只在当前的shell中生效，而环境变量会在当前shell和这个shell的所有子shell当中生效。如果把环境变量写入相应的配置文件，那么这个环境变量就会在所有的shell中生效。

设置环境变量

`export 变量名=变量值 #申明变量`

`env #查询变量`

`unset #删除变量`

3.2.3 位置参数变量

变
量

作用

`$n` `n`为数字, `$0`代表命令本身, `$1-$9`, 代表第一个参数到第九个参数, 10以上的参数需要用大括号包含, 例如`${10}`

`$*` 这个变量代表命令行中所有的参数, `$*`把所有的参数看成一个整体

`$@` 这个变量也代表命令行中所有的参数, `$@`把每个参数区分对待

`$#` 这个变量代表命令行中所有参数的个数

3.2.4 预定义变量

变
量

作用

`$?` 最后一次执行的命令的返回状态。如果这个变量的值为0, 证明上一个命令正确执行; 如果这个变量的值为非0, 则证明上一个命令执行不正确。

`$$` 当前进程的进程号 (PID)

`!` 后台进行的最后一个进程的进程号 (PID)

接收键盘输入

`read [选项] [变量名]`

- `-p`“提示信息”: 在等待`read`输入时, 输出提示信息
- `-t` 秒数: `read`命令会一直等待用户输入, 使用此选项可以指定等待时间
- `-n` 字符数: `read`命令只接受指定的字符数, 就会执行
- `-s`: 隐藏输入的数据, 适用于机密信息的输入

3.3 Bash的运算符

3.3.1 数值运算与运算符

`declare`

`declare [] [+/-] [选项] 变量名`

- `-:` 给变量设定类型属性
- `+`: 取消变量的类型属性

- -i: 将变量声明为整数型
- -x: 将变量声明为环境变量
- -p: 显示指定变量的被声明的类型

例子:

```
aa=11
bb=22
declare -i cc=$aa+$bb
```

expr或let数值运算工具

```
dd=$(expr $aa + $bb)
```

“\$((运算式))”或“\${运算式}”

```
ff=$(( $aa+$bb ))
```

3.3.2 变量测试与内容替换

变量测试（不推荐，太复杂）

变量置换方式	变量y没有设置	变量y为空值	变量y设置值
x=\${y-新值}	x=新值	x为空	x=\$y
x=\${y:-新值}	x=新值	x=新值	x=\$y
x=\${y+新值}	x为空	x=新值	x=新值
x=\${y:+新值}	x为空	x为空	x=新值
x=\${y=新值}	x=新值 y=新值	x为空 y值不变	x=\$y y值不变
x=\${y:=新值}	x=新值 y=新值	x=新值 y=新值	x=\$y y值不变
x=\${y?新值}	新值输出到标准 错误输出（就是 屏幕）	x为空	x=\$y
x=\${y:?新值}	新值输出到标准 错误输出	新值输出到标准 错误输出	x=\$y

3.4 正则表达式

正则表达式用来文件中匹配符合条件的字符串，正则则是包含匹配。grep、awk、sed等命令可以支持正则表达式。

通配符用来匹配符合条件的文件名，通配符是完全匹配。ls、find、cp这些命令不支持正则表达式，所以只能使用shell自己的通配符来进行匹配了。

元字符	作用
*	前一个字符匹配0次或任意多次。
.	匹配除了换行符外任意一个字符。
^	匹配行首。例如：^hello会匹配以hello开头的行。
\$	匹配行尾。例如：hello&会匹配以hello结尾的行。
[]	匹配中括号中指定的任意一个字符，只匹配一个字符。例如：[aoeiu] 匹配任意一个元音字母，[0-9] 匹配任意一位数字，[a-z][0-9]匹配小写字母和一位数字构成的两位字符。
[^]	匹配除中括号的字符以外的任意一个字符。例如：[^0-9] 匹配任意一位非数字字符，[^a-z]表示任意一位非小写字母。
\	转义符。用于取消讲特殊符号的含义取消。
\{n\}	表示其前面的字符恰好出现n次。例如：[0-9]\{4\} 匹配4位数字，[1][3-8][0-9]\{9\}匹配手机号码。
\{n,\}	表示其前面的字符出现不小于n次。例如：[0-9]\{2,\} 表示两位及以下的数字。
\{n,m\}	表示其前面的字符至少出现n次，最多出现m次。例如：[a-z]\{6,8\} 匹配6到8位的小写字母。

3.5 字符截取命令

cut

功能描述：截取符合条件的列

cut [选项] 文件名

- -f 列名：提取第几列
- -d 分隔符：按照指定分隔符分割列

cut局限：如果文件分隔符没有任何规律，则不适用

printf

功能描述：awk命令的最基本输出方法

printf '输出类型输出格式' 输出内容

输出类型：

- %ns: 输出字符串。n是数字指代输出几个字符。
- %ni: 输出整数。n是数字指代输出几个数字。
- %m.nf: 输出浮点数。m和n是数字，指代输出的整数位数和小数位数。如%8.2f代表共输出8位数，其中2位小数，6位整数。

输出格式：

- \a: 输出警告声音
- \b: 输出退格键，也就是Backspace键
- \f: 清楚屏幕
- \n: 换行
- \r: 回车，也就是Enter键
- \t: 水平输出退格键，也就是Tab键
- \v: 垂直输出退格键，也就是Tab键

awk

功能描述：截取符合条件的列。功能强大但是较为复杂。

awk '条件1{动作1} 条件2{动作2}...' 文件名

条件1: BEGIN, 先执行

条件2: END, 最后执行

sed

功能描述：轻量级流编辑器。用来将数据进行选取、替换、删除、新增的命令

vim只能修改文件的内容，不能修改命令结果的内容，而sed可以。

sed [选项] '[动作]' 文件名

选项：

- -n: 一般sed命令会把所有数据都输出到屏幕，如果加入此选项，则只会把经过sed命令处理的行输出到屏幕。
- -e: 允许对输出数据应用多条sed命令编辑
- -i: 用sed的修改结果直接修改读取数据的文件，而不是由屏幕输出

动作：

- a\：追加，在当前行后添加一行或多行。添加多行时，除最后一行外，每行末尾需要用“\”代表数据未完结。
- c\：行替换，用c后面的字符串替换原数据行，替换多行时，除最后一行外，每行末尾需用“\”代表数据未完结。
- i\：插入，在当期行前插入一行或多行。插入多行时，除最后一行外，每行末尾需要用“\”代表数据未完结。
- d: 删除，删除指定的行。
- p: 打印，输出指定的行。
- s: 字符串替换，用一个字符串替换另外一个字符串。格式为“行范围s/旧字符串/新字符串/g”（和vim中的替换格式类似）

3.6 字符处理命令

sort

sort [选项] 文件名

- -f: 忽略大小写
- -n: 以数值型进行排序，默认使用字符串型排序
- -r: 反向排序
- -t: 指定分隔符，默认的分隔符是制表符
- -k n[,m]: 按照指定的字段范围排序。从第n字段开始，m字段结束（默认为行尾）

WC

wc [选项] 文件名

- -l: 只统计行数
- -w: 只统计单词数
- -m: 只统计字符数

3.7 条件判断式

3.7.1 按照文件类型进行判断

测试选项	作 用
-b 文件	判断该文件是否存在，并且是否为块设备文件（是块设备文件为真）
-c 文件	判断该文件是否存在，并且是否为字符设备文件（是字符设备文件为真）
-d 文件	判断该文件是否存在，并且是否为目录文件（是目录为真）
-e 文件	判断该文件是否存在（存在为真）
-f 文件	判断该文件是否存在，并且是否为普通文件（是普通文件为真）
-L 文件	判断该文件是否存在，并且是否为符号链接文件（是符号链接文件为真）
-p 文件	判断该文件是否存在，并且是否为管道文件（是管道文件为真）
-s 文件	判断该文件是否存在，并且是否为非空（非空为真）
-S 文件	判断该文件是否存在，并且是否为套接字文件（是套接字文件为真）

两种判断格式：

```
test -e /root/install.log
```

```
[ -e /root/install.log ]
```

例子：

```
[-d /root] && echo "yes" || echo "no"
```

3.7.2 按照文件权限进行判断

测试选项	作 用
-r 文件	判断该文件是否存在，并且是否该文件拥有读权限（有读权限为真）
-w 文件	判断该文件是否存在，并且是否该文件拥有写权限（有写权限为真）
-x 文件	判断该文件是否存在，并且是否该文件拥有执行权限（有执行权限为真）
-u 文件	判断该文件是否存在，并且是否该文件拥有SUID权限（有SUID权限为真）
-g 文件	判断该文件是否存在，并且是否该文件拥有SGID权限（有SGID权限为真）
-k 文件	判断该文件是否存在，并且是否该文件拥有SBit权限（有SBit权限为真）

3.7.3 两个文件之间进行比较

测试选项	作 用
文件1 -nt 文件2	判断文件1的修改时间是否比文件2的新（如果新则为真）
文件1 -ot 文件2	判断文件1的修改时间是否比文件2的旧（如果旧则为真）
文件1 -ef 文件2	判断文件1是否和文件2的Inode号一致，可以理解为两个文件是否为同一个文件。这个判断用于判断硬链接是很好方法

3.7.4 两个整数之间比较

测试选项	作 用
整数1 -eq 整数2	判断整数1是否和整数2相等（相等为真）
整数1 -ne 整数2	判断整数1是否和整数2不相等（不相等位置）
整数1 -gt 整数2	判断整数1是否大于整数2（大于为真）
整数1 -lt 整数2	判断整数1是否小于整数2（小于位置）
整数1 -ge 整数2	判断整数1是否大于等于整数2（大于等于为真）
整数1 -le 整数2	判断整数1是否小于等于整数2（小于等于为真）

3.7.5 字符串的比较

测试选项	作 用
-z 字符串 +	判断字符串是否为空（为空返回真）
-n 字符串	判断字符串是否为非空（非空返回真）
字串1==字串2	判断字符串1是否和字符串2相等（相等返回真）
字串1!= 字串2	判断字符串1是否和字符串2不相等（不相等返回真）

3.7.6 多重条件判断

测试选项	作 用
判断1 -a 判断2	逻辑与，判断1和判断2都成立，最终的结果才为真
判断1 -o 判断2	逻辑或，判断1和判断2有一个成立，最终的结果就为真
! 判断	逻辑非，使原始的判断式取反

3.8 流程控制

3.8.1 if语句

单分支if条件语句

```
if [ 条件判断式 ];then
• 程序
fi
```

或者

```
if [ 条件判断式 ]
• then
• 程序
fi
```

例子：判断分区使用率

```
# !/bin/bash
#
rate=$(df -h | grep "/dev/sda3" | awk '{print $5}' | cut -d "%" -f 1)
if [ $rate -ge 80 ]
then
    echo "warning! /dev/sda3 is full!!"
fi
```

例子：备份mysql数据库

```
# !/bin/bash
#
ntpdate asia.pool.ntp.org &> /dev/null
# 同步系统时间
date=$(date +%y%m%d)
# 把当前系统时间按照“年月日”格式赋予变量date
size=$(du -sh /var/lib/mysql)
# 统计mysql数据库的大小，并把大小赋予size变量
if [ -d /tmp/dbbak ]
then
    echo "Date: $date!" > /tmp/dbbak/dbinfo.txt
    echo "Date size: $size" >> /tmp/dbbak/dbinfo.txt
    cd /tmp/dbbak
    tar -zcf mysql-lib-$date.tar.gz /var/lib/mysql
    dbinfo.txt &>/dev/null
    rm -rf /tmp/dbbak/dbinfo.txt
else
    mkdir /tmp/dbbak
    echo "Date: $date!" > /tmp/dbbak/dbinfo.txt
    echo "Date size: $size" >> /tmp/dbbak/dbinfo.txt
    cd /tmp/dbbak
    tar -zcf mysql-lib-$date.tar.gz /var/lib/mysql
    dbinfo.txt &>/dev/null
    rm -rf /tmp/dbbak/dbinfo.txt
```

3.8.2 case语句

```
case $变量名 in
    "值1")
        如果变量的值等于值1，则执行程序1
        ;;
    "值2")
        如果变量的值等于值2，则执行程序2
        ;;
    ...省略其他分支...
    *)
        如果变量的值都不是以上的值，则执行此程序
        ;;
esac
```

3.8.3 for循环

语法一

```
for 变量 in 值1 值2 值3...
do
    程序
done
```

例子

```
#!/bin/bash
# 批量解压缩脚本

cd /lamp
ls *.tar.gz > ls.log
for i in $(cat ls.log)
do
    tar -zxf $i &>/dev/null
done
rm -rf /lamp/ls.log
```

语法二

```
for ((初始值;循环控制条件;变量变化))
do
    程序
done
```

例子

```
#!/bin/bash
# 从1加到100
s = 0
for (( i=1;i<=100;i=i+1))
do
    s=$((s+i))
done
```

3.8.4 while循环与util循环

while循环

```
while [条件判断式]
do
    程序
done
```

util循环

```
util [条件判断式]
```

```
do
```

```
    程序
```

```
done
```